🔐 Authentication & User Management

1. Register User

POST /api/v1/users/register
Auth: None
Form-data (multipart):

- firstName (string), lastName (string), email (string), mobile (string), password (string)
- role (optional, default "user" – allowed: user, vendor, driver)
- drivingLicense (file, optional – image)

Response (201):

json


```
{
  "user": {
    "_id": "65f1a2b3c4d5e6f7a8b9c0d1",
    "firstName": "John",
    "lastName": "Doe",
    "email": "john@example.com",
    "mobile": "9876543210",
    "role": "user",
    "avatar": null,
    "drivingLicense": "/licenses/license-1700000000000-123456789.jpeg"
  },
  "status": "success",
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```


---

2. Login (Email & Password)

POST /api/v1/users/login
Auth: None
Body (JSON):

json

```
{
  "email": "john@example.com",
  "password": "mypassword"
}
```

Response (200):
Same as register response (user + token).

---

3. Login via Mobile (Send OTP)

POST /api/v1/users/login-mobile
Auth: None
Body:

json

```
{ "mobile": "9876543210" }
```

Response (200):

json

```
{
  "status": "success",
  "message": "OTP sent successfully!",
  "otp": 123456
}
```

⚠️ In production, remove otp from response.

---

4. Verify Mobile OTP & Login

POST /api/v1/users/verify-mobile-otp
Auth: None
Body:

json

```
{
  "mobile": "9876543210",
  "otp": 123456
}
```

Response (200): User object + token (same as register).

---

5. Logout

POST /api/v1/users/logout
Auth: Bearer token or cookie
Response (200):

json

```
{ "status": "success" }
```

---

6. Update Password (when logged in)

POST /api/v1/users/update-password
Auth: Required
Body:

json

```
{
  "currentPassword": "oldpass",
  "newPassword": "newpass123"
}
```

Response (200):

json

{

"status": "success",

"message": "Password updated successfully!"

}

👤 User Profile

7. Get Current User Profile

GET /api/v1/users/me
Auth: Required
Response (200):

json

{

"status": "success",

"data": { "user": { "_id": "...", "firstName": "John", ... } }

}

8. Update Current User (basic info)

PATCH /api/v1/users/me
Auth: Required
Body (any of): firstName, lastName, gender, pincode, city, state, country
Response (200):

json

{

"status": "success",

"message": "Profile updated successfully!"

}

9. Upload Profile Picture

POST /api/v1/users/me/upload-image
Auth: Required

Form-data: avatar (image file)
Response (200):

json

```
{
  "status": "success",
  "message": "Profile image uploaded successfully!",
  "data": { "imagePath": "/profile/profile-65f1a2b3...-123456789.jpg" }
}
```

👨‍✈️ Driver Management (Vendor only)

10. Add a Driver

POST /api/v1/users/add-driver
Auth: Required + role: "vendor"
Body (JSON):

json

```
{
  "fullName": "Driver Name",
  "email": "driver@example.com",
  "mobile": "9988776655",
  "password": "driverpass",
  "drivingLicenseNumber": "DL123456",
  "aadharNumber": "111122223333"
}
```

Response (201):

json

```
{
  "status": "success",
```

"message": "Driver added successfully!",

"data": { "driver": { ... } }

}

11. Get All Drivers Added by Me (Vendor)

GET /api/v1/users/vendor-drivers
Auth: Required + vendor
Response (200):

json

{

"status": "success",

"data": { "drivers": [...] }

}

12. Delete a Driver (Vendor only)

DELETE /api/v1/users/driver/:driverId
Auth: Required + vendor
Response (200):

json

{

"status": "success",

"message": "Driver deleted successfully"

}

13. Update a Driver (Vendor only)

PUT /api/v1/users/driver/:driverId
Auth: Required + vendor
Body (any of): fullName, email, mobile, drivingLicenseNumber, aadharNumber, password
Response (200):

json

{

```
  "status": "success",
  "message": "Driver updated successfully",
  "data": { "driver": {...} }
}
```

🛠️ Machinery (Equipment) Endpoints

14. Get All Machinery (Public)

GET /api/v1/machinery/
Auth: None
Response (200):

```
json
{
  "status": "success",
  "results": 10,
  "data": { "machinery": [...] }
}
```

15. Get Single Machinery by Slug

GET /api/v1/machinery/:id
Auth: None
Response (200):

```
json
{
  "status": "success",
  "data": { "machinery": { ... } }
}
```

16. Get My Machinery (Vendor)

GET /api/v1/machinery/me
Auth: Required (vendor)
Response (200):

json

```
{
  "status": "success",
  "results": 3,
  "data": { "machinery": [...] }
}
```

17. Add New Machinery

POST /api/v1/machinery/
Auth: Required (vendor)
Form-data (multipart):

- name, description, category, baseHourlyRate, dailyCapPrice, weeklyCapPrice, monthlyCapPrice
- latitude, longitude, address, city, state, pincode
- images[] (multiple image files)

Response (201):

json

```
{
  "status": "success",
  "message": "Machinery added successfully!",
  "data": { "machinery": { ... } }
}
```

18. Edit Machinery (Admin only)

PATCH /api/v1/machinery/:id
Auth: Required + admin

Body (JSON): any updatable fields
Response (200):

json

```
{
  "status": "success",
  "message": "Machinery updated successfully!"
}
```

📅 Bookings

19. Create a Booking

POST /api/v1/bookings/
Auth: Required
Body (JSON):

json

```
{
  "vehicleId": "65f1a2b3c4d5e6f7a8b9c0d1",
  "startDateTime": "2025-06-01T10:00:00Z",
  "endDateTime": "2025-06-03T18:00:00Z",
  "location": "Pickup address"
}
```

Response (201):

json

```
{
  "success": true,
  "data": {
    "_id": "...",
    "user": "...",
```

```
    "vehicle": "...",
    "startDateTime": "...",
    "endDateTime": "...",
    "totalHours": 56,
    "totalAmount": 2800,
    "status": "pending"
  }
}
```

20. Get My Bookings (Logged-in User)

GET /api/v1/bookings/my-bookings
Auth: Required
Response (200):

```
json
{
  "success": true,
  "data": [ { booking populated with vehicle & driver } ]
}
```

21. Get Vendor Bookings (Vendor only)

GET /api/v1/bookings/vendor-bookings
Auth: Required + vendor
Response (200): List of bookings where vehicle belongs to this vendor.

22. Get Driver Bookings (Driver only)

GET /api/v1/bookings/driver-bookings
Auth: Required + driver
Response (200): Bookings assigned to this driver.

23. Assign Driver to Booking

PATCH /api/v1/bookings/assign-driver/:bookingId
Auth: Required (usually vendor/admin)
Body:

json

{ "driverId": "driver_user_id" }

Response (200):

json

{

"success": true,

"message": "Driver assigned successfully",

"data": { ...booking }

}

24. Update Booking Status

PATCH /api/v1/bookings/update-status/:bookingId
Auth: Required
Body:

json

{ "status": "confirmed" } // allowed: pending, confirmed, driver_assigned, started, on_the_way, reached, pending_payment, completed, cancelled

Response (200):

json

{

"success": true,

"message": "Booking status updated",

"data": { ...booking }

}

25. Get Single Booking

GET /api/v1/bookings/:id
Auth: Required
Response (200): Full booking with populated vehicle & user.

26. Get All Bookings (Admin only)

GET /api/v1/bookings/
Auth: Required + admin
Response (200):

json

{

"success": true,

"data": [...all bookings]

}

👑 **Admin Only Endpoints**

27. Get All Users

GET /api/v1/users/
Auth: Required + admin
Response (200): List of all users.

28. Get Single User by ID

GET /api/v1/users/:id
Auth: Required + admin

29. Delete User

DELETE /api/v1/users/:id
Auth: Required + admin

30. Get All Drivers (Admin)

GET /api/v1/users/drivers
Auth: Required + admin
Response: List of all users with role: "driver".

API Documentation - Ekcors

Authentication & User Management

1. Register User

`POST /api/v1/users/register`

- **Auth:** None

- **Form-data:** firstName, lastName, email, mobile, password, role (user/vendor/driver), drivingLicense (file)

2. Login (Email & Password)

`POST /api/v1/users/login`

- **Body:** `{ "email": "...", "password": "..." }`

3. Login via Mobile (Send OTP)

`POST /api/v1/users/login-mobile`

- **Body:** `{ "mobile": "9876543210" }`

4. Verify Mobile OTP

`POST /api/v1/users/verify-mobile-otp`

- **Body:** `{ "mobile": "...", "otp": 123456 }`

5. Logout

`POST /api/v1/users/logout` (Auth required)

6. Update Password

`POST /api/v1/users/update-password` (Auth required)

- **Body:** `{ "currentPassword": "...", "newPassword": "..." }`

User Profile

7. Get Current User

`GET /api/v1/users/me` (Auth required)

8. Update Profile

`PATCH /api/v1/users/me`

- **Body:** firstName, lastName, gender, pincode, city, state, country

9. Upload Profile Picture

`POST /api/v1/users/me/upload-image` (Auth required, form-data: avatar)

Driver Management (Vendor only)

10. Add Driver

`POST /api/v1/users/add-driver`

- **Body:** fullName, email, mobile, password, drivingLicenseNumber, aadharNumber

11. Get My Drivers

`GET /api/v1/users/vendor-drivers`

12. Delete Driver

`DELETE /api/v1/users/driver/:driverId`

13. Update Driver

`PUT /api/v1/users/driver/:driverId`

Machinery (Equipment)

14. Get All Machinery

`GET /api/v1/machinery/` (Public)

15. Get Single Machinery

`GET /api/v1/machinery/:id` (Public)

16. Get My Machinery (Vendor)

`GET /api/v1/machinery/me` (Auth required)

17. Add Machinery

`POST /api/v1/machinery/` (Vendor, form-data: name, description, baseHourlyRate, dailyCapPrice, etc., images[])

18. Edit Machinery

`PATCH /api/v1/machinery/:id` (Admin only)

Bookings

19. Create Booking

`POST /api/v1/bookings/` (Auth required)

- **Body:** `{ "vehicleId": "...", "startDateTime": "...", "endDateTime": "...", "location": "..." }`

20. My Bookings

`GET /api/v1/bookings/my-bookings` (Auth required)

21. Vendor Bookings

`GET /api/v1/bookings/vendor-bookings` (Vendor)

22. Driver Bookings

`GET /api/v1/bookings/driver-bookings` (Driver)

23. Assign Driver

`PATCH /api/v1/bookings/assign-driver/:bookingId`

- **Body:** `{ "driverId": "..." }`

24. Update Booking Status

`PATCH /api/v1/bookings/update-status/:bookingId`

- **Body:** `{ "status": "confirmed" }` (pending, confirmed, driver_assigned, started, on_the_way, reached, pending_payment, completed, cancelled)

25. Get Single Booking

`GET /api/v1/bookings/:id` (Auth required)

26. All Bookings (Admin)

`GET /api/v1/bookings/` (Admin)

Admin Only

27. Get All Users

`GET /api/v1/users/` (Admin)

28. Get User by ID

`GET /api/v1/users/:id` (Admin)

29. Delete User

`DELETE /api/v1/users/:id` (Admin)

30. Get All Drivers (Admin)

`GET /api/v1/users/drivers` (Admin)